UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA EN CIENCIAS Y SISTEMAS
MANEJO E IMPLEMENTACIÓN DE ARCHIVOS
SEGUNDO SEMESTRE 2026
PROYECTO 1

Andy Jefferson González Fuentes 202231338

MANUAL TÉCNICO

TABLA DE CONTENIDO

Alcance funcional **1**
Diagrama de despliegue **2**
Diagramas complementarios **2**
Estructura de paquetes **6**
Modelo de datos **6**
Relaciones principales **7**
Integridad y control de stock **7**
Autenticación con JWT **8**
Autorización basada en roles **8**
Autenticación /api/auth 9
Usuario autenticado /api/usuarios 9
Administración de usuarios /api/admin/usuarios 9
Áreas / Roles /api/admin/areas 9
Categorías /api/admin/categorias 10
Productos /api/admin/productos 10
Proveedores /api/admin/proveedores 10
Clientes /api/admin/clientes 11
Compras /api/admin/compras 11
Ventas /api/admin/ventas 11
Inventario /api/admin/inventario 12
Auditoría /api/admin/auditoria 12
Dashboard /api/admin/dashboard 12
Reportes /api/admin/reportes 13
Migraciones de base de datos **14**

INTRODUCCIÓN

Este manual describe la arquitectura, el modelo de datos, los módulos funcionales y la configuración técnica del backend del sistema Mini ERP. Su propósito es servir como referencia para el equipo de desarrollo y para cualquier persona encargada de mantener, desplegar o extender el sistema.

El backend expone una API REST consumida por un cliente web desarrollado en Angular (SPA), y persiste su información en una base de datos PostgreSQL administrada mediante migraciones versionadas con Flyway.

Alcance funcional

El sistema cubre los siguientes procesos de negocio:

- Gestión de usuarios, áreas y roles de acceso.
- Catálogo de categorías y productos, con control de stock mínimo.
- Gestión de proveedores y registro de compras (entradas de inventario).
- Gestión de clientes y registro de ventas, incluida la generación de facturas en PDF.
- Control de inventario (kardex) mediante movimientos de entrada y salida.
- Auditoría de las acciones realizadas por los usuarios.
- Reportes gerenciales y panel de indicadores (dashboard).

ARQUITECTURA DEL SISTEMA

El backend sigue una arquitectura en capas típica de una aplicación Spring Boot:

- Controller: expone los endpoints REST y valida las peticiones de entrada (DTOs).
- Service: contiene la lógica de negocio (cálculo de totales, control de stock, reglas de estado, etc.).
- Repository: acceso a datos mediante Spring Data JPA sobre las entidades del dominio.
- Entity: modelo de datos persistente, mapeado a las tablas de PostgreSQL.
- DTO: objetos de transferencia usados en las peticiones y respuestas de la API, para no exponer las entidades directamente.
- Config / Security: configuración de Spring Security, CORS y el filtro de autenticación JWT.

Diagrama de despliegue

La arquitectura física del sistema está compuesta por tres nodos: el equipo cliente (navegador con el frontend Angular), el servidor de aplicaciones (backend Spring Boot, puerto 8080) y el servidor de base de datos (PostgreSQL, puerto 5432). El cliente se comunica con el backend

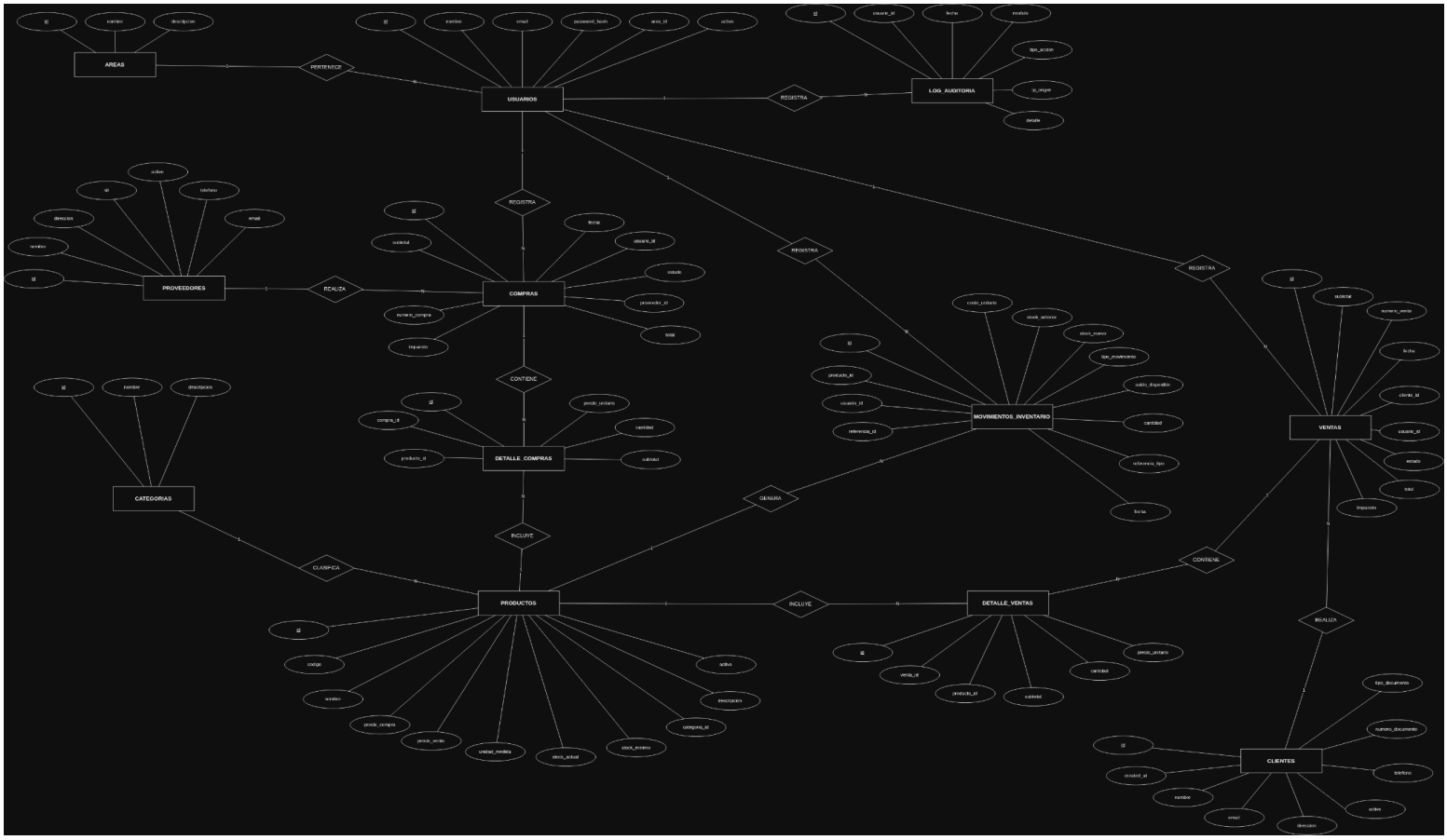
vía HTTP/HTTPS usando JSON y un token JWT (Bearer), y el backend se comunica con la base de datos vía JDBC.

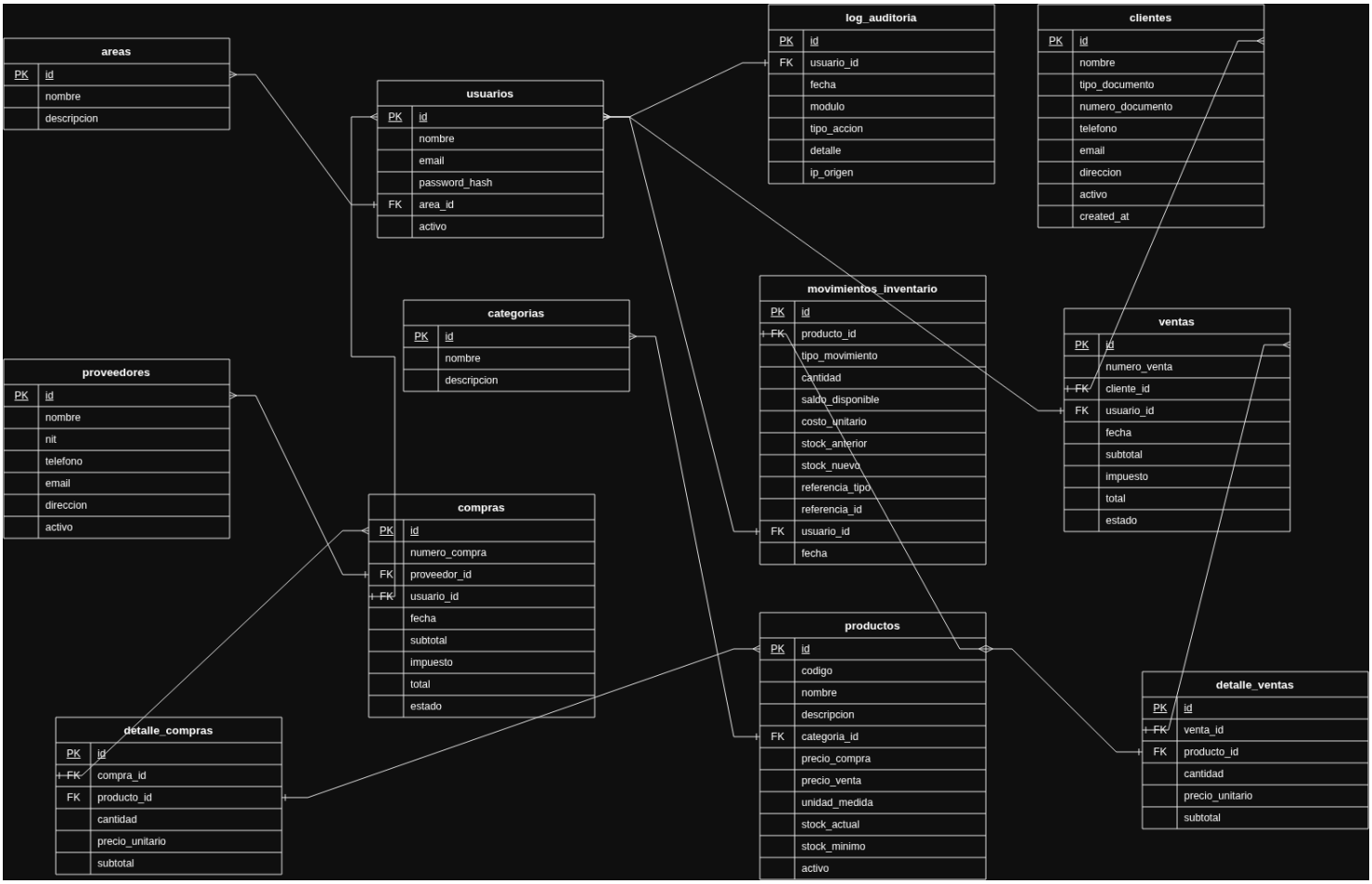


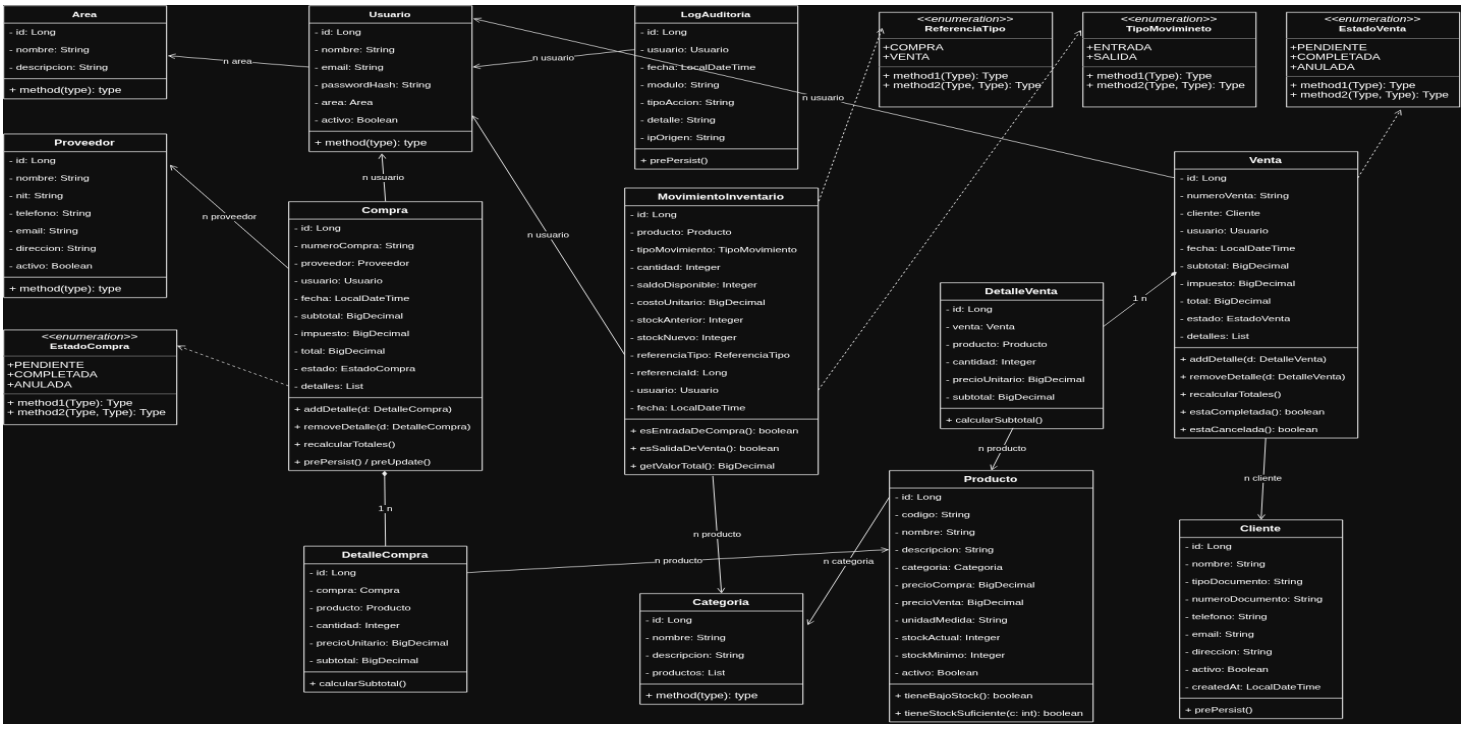
Diagramas complementarios

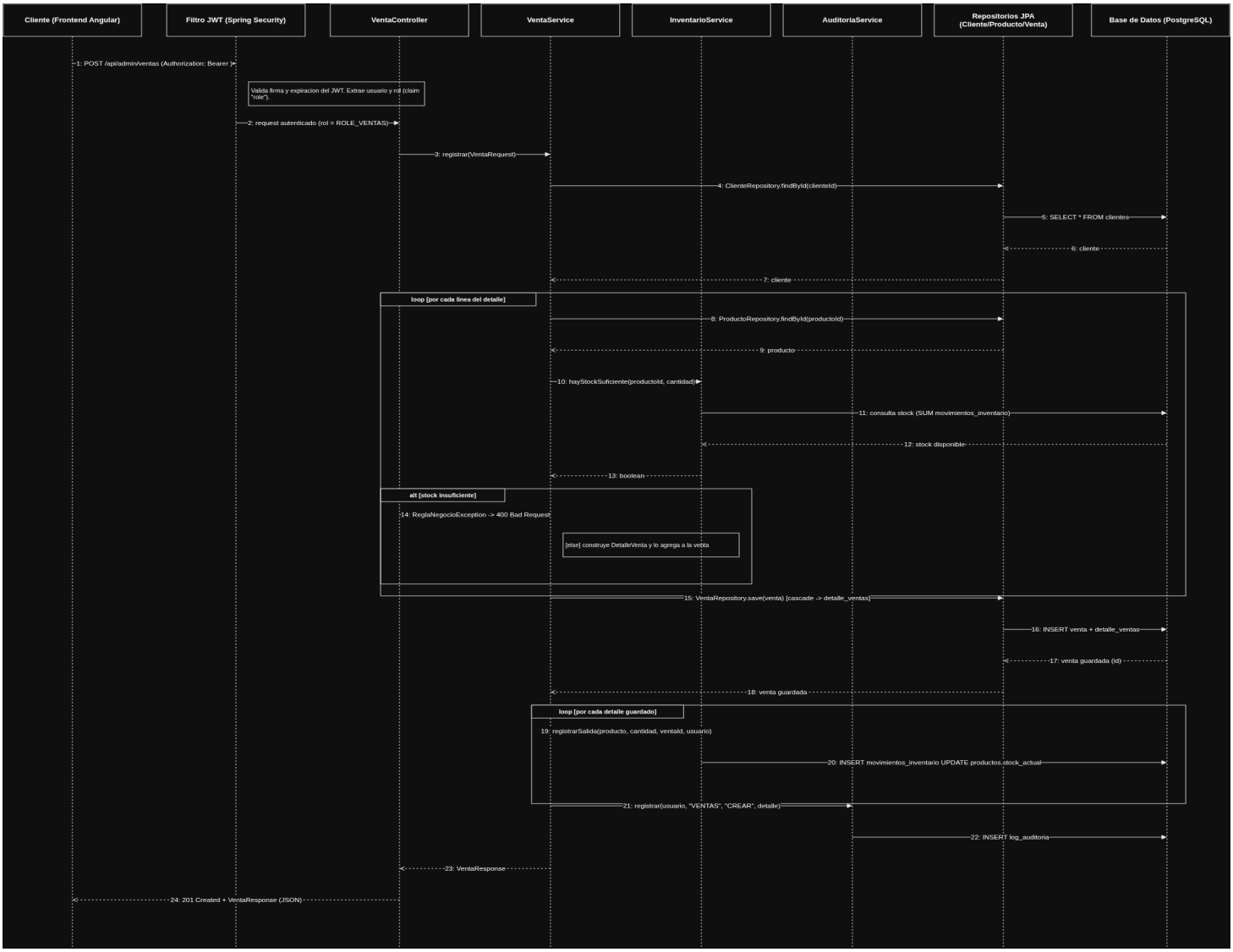
Junto con este manual se entregan por separado los siguientes diagramas en formato .drawio:

- Modelo entidad-relación conceptual (notación de Chen).
- Diagrama físico de tablas de la base de datos MySQL/PostgreSQL.
- Diagrama de clases del backend.
- Diagrama de despliegue.




areas
PK id
nombre
descripcion
usuarios
PK id
nombre
email
password_hash
FK area_id
activo
log_auditoria
PK id
FK usuario_id
fecha
modulo
tipo_accion
detalle
ip_origen
clientes
PK id
nombre
tipo_documento
numero_documento
telefono
email
direccion
activo
created_at
categorias
PK id
nombre
descripcion
movimientos_inventario
PK id
FK producto_id
tipo_movimiento
cantidad
saldo_disponible
costo_unitario
stock_anterior
stock_nuevo
referencia_tipo
referencia_id
FK usuario_id
fecha
ventas
PK id
numero_venta
FK cliente_id
FK usuario_id
fecha
subtotal
impuesto
total
estado
proveedores
PK id
nombre
nit
telefono
email
direccion
activo
compras
PK id
numero_compra
FK proveedor_id
FK usuario_id
fecha
subtotal
impuesto
total
estado
productos
PK id
codigo
nombre
descripcion
FK categoria_id
precio_compra
precio_venta
unidad_medida
stock_actual
stock_minimo
activo
detalle_ventas
PK id
FK venta_id
FK producto_id
cantidad
precio_unitario
subtotal
detalle_compras
PK id
FK compra_id
FK producto_id
cantidad
precio_unitario
subtotal


Area
- id: Long
- nombre: String
- descripcion: String
+ method(type): type
Usuario
- id: Long
- nombre: String
- email: String
- passwordHash: String
- area: Area
- activo: Boolean
+ method(type): type
LogAuditoria
- id: Long
- usuario: Usuario
- fecha: LocalDateTime
- modulo: String
- tipoAccion: String
- detalle: String
- ipOrigen: String
+ prePersist()
<<enumeration>>
ReferenciaTipo
+COMPRA
+VENTA
+ method1(Type): Type
+ method2(Type, Type): Type
<<enumeration>>
TipoMovimiento
+ENTRADA
+SALIDA
+ method1(Type): Type
+ method2(Type, Type): Type
<<enumeration>>
EstadoVenta
+PENDIENTE
+COMPLETADA
+ANULADA
+ method1(Type): Type
+ method2(Type, Type): Type
Proveedor
- id: Long
- nombre: String
- nit: String
- telefono: String
- email: String
- direccion: String
- activo: Boolean
+ method(type): type
Compra
- id: Long
- numeroCompra: String
- proveedor: Proveedor
- usuario: Usuario
- fecha: LocalDateTime
- subtotal: BigDecimal
- impuesto: BigDecimal
- total: BigDecimal
- estado: EstadoCompra
- detalles: List
+ addDetalle(d: DetalleCompra)
+ removeDetalle(d: DetalleCompra)
+ recalcularTotales()
+ prePersist() / preUpdate()
MovimientoInventario
- id: Long
- producto: Producto
- tipoMovimiento: TipoMovimiento
- cantidad: Integer
- saldoDisponible: Integer
- costoUnitario: BigDecimal
- stockAnterior: Integer
- stockNuevo: Integer
- referenciaTipo: ReferenciaTipo
- referenciaId: Long
- usuario: Usuario
- fecha: LocalDateTime
+ esEntradaDeCompra(): boolean
+ esSalidaDeVenta(): boolean
+ getValorTotal(): BigDecimal
Venta
- id: Long
- numeroVenta: String
- cliente: Cliente
- usuario: Usuario
- fecha: LocalDateTime
- subtotal: BigDecimal
- impuesto: BigDecimal
- total: BigDecimal
- estado: EstadoVenta
- detalles: List
+ addDetalle(d: DetalleVenta)
+ removeDetalle(d: DetalleVenta)
+ recalcularTotales()
+ estaCompletada(): boolean
+ estaCancelada(): boolean
DetalleVenta
- id: Long
- venta: Venta
- producto: Producto
- cantidad: Integer
- precioUnitario: BigDecimal
- subtotal: BigDecimal
+ calcularSubtotal()
<<enumeration>>
EstadoCompra
+PENDIENTE
+COMPLETADA
+ANULADA
+ method1(Type): Type
+ method2(Type, Type): Type
DetalleCompra
- id: Long
- compra: Compra
- producto: Producto
- cantidad: Integer
- precioUnitario: BigDecimal
- subtotal: BigDecimal
+ calcularSubtotal()
Producto
- id: Long
- codigo: String
- nombre: String
- descripcion: String
- categoria: Categoria
- precioCompra: BigDecimal
- precioVenta: BigDecimal
- unidadMedida: String
- stockActual: Integer
- stockMinimo: Integer
- activo: Boolean
+ tieneBajoStock(): boolean
+ tieneStockSuficiente(c: int): boolean
Categoria
- id: Long
- nombre: String
- descripcion: String
- productos: List
+ method(type): type
Cliente
- id: Long
- nombre: String
- tipoDocumento: String
- numeroDocumento: String
- telefono: String
- email: String
- direccion: String
- activo: Boolean
- createdAt: LocalDateTime
+ prePersist()
n area
n usuario
n usuario
n usuario
n usuario
n proveedor
n usuario
1 n
n producto
n producto
n producto
n categoria
1 n
n cliente


Cliente (Frontend Angular)
Filtro JWT (Spring Security)
VentaController
VentaService
InventarioService
AuditoriaService
Repositorios JPA (Cliente/Producto/Venta)
Base de Datos (PostgreSQL)
1: POST /api/admin/ventas (Authorization: Bearer)
Valida firma y expiracion del JWT. Extrae usuario y rol (claim "role").
2: request autenticado (rol = ROLE_VENTAS)
3: registrar(VentaRequest)
4: ClienteRepository.findById(clienteId)
5: SELECT * FROM clientes
6: cliente
7: cliente
loop [por cada linea del detalle]
8: ProductoRepository.findById(productoId)
9: producto
10: hayStockSuficiente(productoId, cantidad)
11: consulta stock (SUM movimientos_inventario)
12: stock disponible
13: boolean
alt [stock insuficiente]
14: ReglaNegocioException -> 400 Bad Request
[else] construye DetalleVenta y lo agrega a la venta
15: VentaRepository.save(venta) [cascade -> detalle_ventas]
16: INSERT venta + detalle_ventas
17: venta guardada (id)
18: venta guardada
loop [por cada detalle guardado]
19: registrarSalida(producto, cantidad, ventaId, usuario)
20: INSERT movimientos_inventario UPDATE productos.stock_actual
21: registrar(usuario, "VENTAS", "CREAR", detalle)
22: INSERT log_auditoria
23: VentaResponse
24: 201 Created + VentaResponse (JSON)

TECNOLOGÍAS UTILIZADAS

Componente	Tecnología
Lenguaje	Java (Jakarta EE / Jakarta Servlet API)
Framework backend	Spring Boot (Spring MVC, Spring Data JPA, Spring Security)
Seguridad	Spring Security + autenticación con JWT (JSON Web Token)
Base de datos	PostgreSQL
Migraciones de BD	Flyway (carpeta db/migration, versionado V1, V2, ...)
Mapeo objeto-relacional	Hibernate / JPA
Generación de código repetitivo	Lombok
Serialización JSON	Jackson
Servidor de aplicación	Tomcat embebido (incluido en Spring Boot)
Frontend (consumidor de la API)	Angular (SPA), ejecutado en http://localhost:4200 en desarrollo

Estructura de paquetes

El código fuente del backend se organiza en el paquete base com.minierp.backend, con las siguientes subcarpetas:

Paquete	Contenido
controller	Controladores REST (uno por módulo funcional).
service / service.impl	Interfaces de servicio y su implementación con la lógica de negocio.
repository	Interfaces de acceso a datos (Spring Data JPA).
entity / entity.enums	Entidades JPA y enumeraciones del dominio.
dto	Objetos de transferencia, agrupados por módulo (usuario, producto, compra, venta, etc.).
config	Configuración de seguridad, CORS y el filtro JWT.
exception	Excepciones de negocio y manejador global de errores.
util	Clases utilitarias.

Modelo de datos

La base de datos está compuesta por doce tablas principales, gestionadas mediante migraciones Flyway. A continuación se resume cada entidad; el detalle completo de columnas,

llaves primarias y foráneas se encuentra en los diagramas entidad-relación y de tablas entregados por separado.

Tabla	Descripción
areas	Áreas de la organización; se utilizan también como catálogo de roles del sistema.
usuarios	Usuarios que acceden al sistema, asociados a un área/rol.
categorias	Categorías del catálogo de productos.
productos	Catálogo de productos, con precios, stock actual y stock mínimo.
proveedores	Proveedores para el proceso de compras.
clientes	Clientes para el proceso de ventas.
compras	Encabezado de las órdenes de compra.
detalle_compras	Líneas de detalle de cada compra.
ventas	Encabezado de las ventas.
detalle_ventas	Líneas de detalle de cada venta.
movimientos_inventario	Kardex: movimientos de entrada y salida de inventario.
log_auditoria	Registro de auditoría de acciones de los usuarios.

Relaciones principales

- Un área agrupa a varios usuarios (1:N).
- Una categoría agrupa a varios productos (1:N).
- Una compra pertenece a un proveedor y a un usuario, y se compone de varias líneas de detalle_compras (composición 1:N).
- Una venta pertenece a un cliente y a un usuario, y se compone de varias líneas de detalle_ventas (composición 1:N).
- Cada línea de detalle (compra o venta) referencia un producto.
- Cada movimiento de inventario referencia un producto y al usuario que lo generó.
- Cada registro de auditoría referencia al usuario que ejecutó la acción.

Integridad y control de stock

El campo stock_actual de productos se mantiene sincronizado a través de los movimientos de inventario: las compras generan movimientos de tipo ENTRADA y las ventas generan movimientos de tipo SALIDA, cada uno con su stock_anterior y stock_nuevo, permitiendo trazabilidad completa (kardex).

SEGURIDAD Y AUTENTICACIÓN

Autenticación con JWT

El acceso a la API se controla mediante JSON Web Tokens. El flujo es el siguiente:

- El cliente envía credenciales (email y contraseña) a POST /api/auth/login.
- El backend valida las credenciales contra la tabla usuarios (contraseña almacenada con BCrypt) y, si son correctas, genera un JWT firmado que incluye el correo del usuario y su rol (claim "role").
- El cliente debe enviar este token en cada petición posterior, en la cabecera Authorization: Bearer <token>.
- Un filtro (JwtAuthenticationFilter), ejecutado una vez por petición, valida la firma y expiración del token y coloca la autenticación en el contexto de seguridad de Spring, sin necesidad de consultar la base de datos en cada request.

Autorización basada en roles

Cada endpoint declara, mediante la anotación @PreAuthorize, los roles autorizados a invocarlo. Los roles disponibles corresponden a las áreas registradas en la tabla areas:

Rol	Área asociada	Enfoque principal
ROLE_ADMINISTRACION	ADMINISTRACION	Supervisión general, configuración y reportes.
ROLE_COMPRAS	COMPRAS	Gestión de proveedores y registro de compras.
ROLE_VENTAS	VENTAS	Gestión de clientes y registro de ventas.
ROLE_INVENTARIO	INVENTARIO	Gestión del catálogo de productos y control de stock.

CORS

El backend habilita CORS únicamente para los orígenes http://localhost:4200 y http://127.0.0.1:4200 (frontend Angular en desarrollo), permitiendo los métodos GET, POST, PUT, PATCH, DELETE y OPTIONS, y exponiendo la cabecera Authorization.

MÓDULOS Y ENDPOINTS DE LA API

Todos los endpoints, salvo el de login, requieren un token JWT válido. La columna "Rol requerido" indica las autoridades aceptadas por @PreAuthorize; "Autenticado" indica que basta con tener sesión válida, sin restricción de rol.

Autenticación /api/auth

Método	Ruta	Rol requerido	Descripción
POST	/api/auth/login	Público	Valida credenciales (email + password) y devuelve un JWT junto con los datos básicos del usuario.

Usuario autenticado /api/usuarios

Método	Ruta	Rol requerido	Descripción
GET	/api/usuarios/me	Autenticado	Devuelve el perfil del usuario actualmente autenticado (a partir del JWT).

Administración de usuarios /api/admin/usuarios

Método	Ruta	Rol requerido	Descripción
GET	/api/admin/usuarios	ADMINISTRACION	Lista todos los usuarios del sistema.
GET	/api/admin/usuarios/{id}	ADMINISTRACION	Obtiene el detalle de un usuario por id.
POST	/api/admin/usuarios	ADMINISTRACION	Crea un nuevo usuario y lo asocia a un área/rol.
PUT	/api/admin/usuarios/{id}	ADMINISTRACION	Actualiza los datos de un usuario.
PUT	/api/admin/usuarios/{id}/password	ADMINISTRACION	Restablece la contraseña de un usuario.
PATCH	/api/admin/usuarios/{id}/activar	ADMINISTRACION	Reactiva un usuario deshabilitado.
PATCH	/api/admin/usuarios/{id}/desactivar	ADMINISTRACION	Deshabilita el acceso de un usuario (baja lógica).

Áreas / Roles /api/admin/areas

Método	Ruta	Rol requerido	Descripción
GET	/api/admin/areas	ADMINISTRACION	Lista las áreas, usadas también como catálogo de roles del sistema.

Categorías /api/admin/categorias

Método	Ruta	Rol requerido	Descripción
GET	/api/admin/categorias	Autenticado	Lista las categorías de productos.
GET	/api/admin/categorias/{id}	Autenticado	Obtiene una categoría por id.
POST	/api/admin/categorias	ADMINISTRACION / INVENTARIO	Crea una nueva categoría.
PUT	/api/admin/categorias/{id}	ADMINISTRACION / INVENTARIO	Actualiza una categoría existente.
DELETE	/api/admin/categorias/{id}	ADMINISTRACION	Elimina una categoría.

Productos /api/admin/productos

Método	Ruta	Rol requerido	Descripción
GET	/api/admin/productos	Autenticado	Lista el catálogo de productos.
GET	/api/admin/productos/{id}	Autenticado	Obtiene el detalle de un producto.
POST	/api/admin/productos	ADMINISTRACION / INVENTARIO	Registra un nuevo producto.
PUT	/api/admin/productos/{id}	ADMINISTRACION / INVENTARIO	Actualiza un producto existente.
PATCH	/api/admin/productos/{id}/activar	ADMINISTRACION	Reactiva un producto.
PATCH	/api/admin/productos/{id}/desactivar	ADMINISTRACION	Desactiva un producto (baja lógica).

Proveedores /api/admin/proveedores

Método	Ruta	Rol requerido	Descripción
GET	/api/admin/proveedores	ADMINISTRACION / COMPRAS	Lista los proveedores registrados.

GET	/api/admin/proveedores/{id}	ADMINISTRACION / COMPRAS	Obtiene el detalle de un proveedor.
POST	/api/admin/proveedores	ADMINISTRACION / COMPRAS	Registra un nuevo proveedor.
PUT	/api/admin/proveedores/{id}	ADMINISTRACION / COMPRAS	Actualiza los datos de un proveedor.
PATCH	/api/admin/proveedores/{id}/activar	ADMINISTRACION	Reactiva un proveedor.
PATCH	/api/admin/proveedores/{id}/desactivar	ADMINISTRACION	Desactiva un proveedor.

Clientes /api/admin/clientes

Método	Ruta	Rol requerido	Descripción
GET	/api/admin/clientes	ADMINISTRACION / VENTAS	Lista los clientes registrados.
GET	/api/admin/clientes/{id}	ADMINISTRACION / VENTAS	Obtiene el detalle de un cliente.
POST	/api/admin/clientes	ADMINISTRACION / VENTAS	Registra un nuevo cliente.
PUT	/api/admin/clientes/{id}	ADMINISTRACION / VENTAS	Actualiza los datos de un cliente.
PATCH	/api/admin/clientes/{id}/activar	ADMINISTRACION	Reactiva un cliente.
PATCH	/api/admin/clientes/{id}/desactivar	ADMINISTRACION	Desactiva un cliente.

Compras /api/admin/compras

Método	Ruta	Rol requerido	Descripción
GET	/api/admin/compras	ADMINISTRACION / COMPRAS	Lista las órdenes de compra.
GET	/api/admin/compras/{id}	ADMINISTRACION / COMPRAS	Obtiene el detalle de una compra con sus líneas.
POST	/api/admin/compras	COMPRAS	Registra una nueva compra y genera movimientos de inventario de entrada.
PATCH	/api/admin/compras/{id}/estado	COMPRAS	Actualiza el estado de la compra (pendiente / completada / anulada).

Ventas /api/admin/ventas

Método	Ruta	Rol requerido	Descripción
GET	/api/admin/ventas	ADMINISTRACION / VENTAS	Lista las ventas registradas.
GET	/api/admin/ventas/{id}	ADMINISTRACION / VENTAS	Obtiene el detalle de una venta con sus líneas.
POST	/api/admin/ventas	VENTAS	Registra una nueva venta, valida stock y genera movimientos de inventario de salida.
GET	/api/admin/ventas/{id}/factura	ADMINISTRACION / VENTAS	Genera y descarga la factura de la venta en formato PDF.

Inventario /api/admin/inventario

Método	Ruta	Rol requerido	Descripción
GET	/api/admin/inventario/stock	Autenticado	Consulta el stock actual de los productos.
GET	/api/admin/inventario/bajo-stock	ADMINISTRACION / INVENTARIO / COMPRAS	Lista los productos cuyo stock está en o por debajo del mínimo.
GET	/api/admin/inventario/movimientos/{productoId}	ADMINISTRACION / INVENTARIO	Muestra el historial (kardex) de movimientos de un producto.

Auditoría /api/admin/auditoria

Método	Ruta	Rol requerido	Descripción
GET	/api/admin/auditoria	ADMINISTRACION	Consulta el registro de auditoría (acciones realizadas por los usuarios).

Dashboard /api/admin/dashboard

Método	Ruta	Rol requerido	Descripción
GET	/api/admin/dashboard/resumen	ADMINISTRACION	Devuelve indicadores generales del sistema (ventas, compras, stock, etc.).

Reportes /api/admin/reportes

Método	Ruta	Rol requerido	Descripción
GET	/api/admin/reportes/productos-mas-vendidos	ADMINISTRACION	Ranking de productos con mayor volumen de venta.
GET	/api/admin/reportes/productos-menor-existencia	ADMINISTRACION	Productos con menor existencia en inventario.
GET	/api/admin/reportes/productos-mas-movimientos	ADMINISTRACION	Productos con mayor cantidad de movimientos de inventario.
GET	/api/admin/reportes/top-proveedores	ADMINISTRACION	Proveedores con mayor volumen de compras.
GET	/api/admin/reportes/productos-mas-adquiridos	ADMINISTRACION	Productos más adquiridos en compras.
GET	/api/admin/reportes/top-clientes	ADMINISTRACION	Clientes con mayor volumen de compra.
GET	/api/admin/reportes/productos-mayores-ingresos	ADMINISTRACION	Productos que generan mayores ingresos por venta.
GET	/api/admin/reportes/resumen-ventas	ADMINISTRACION	Resumen general de ventas en un periodo.

LÓGICA DEL NEGOCIO

- Una compra y una venta se registran junto con todas sus líneas de detalle en una sola operación transaccional (composición Compra–DetalleCompra y Venta–DetalleVenta con cascada y eliminación huérfana).
- Cada línea de compra o venta genera automáticamente un movimiento de inventario (ENTRADA o SALIDA) que actualiza el stock_actual del producto.
- Antes de completar una venta, el sistema valida que exista stock suficiente para cada producto solicitado.
- Un producto se considera con bajo stock cuando stock_actual es menor o igual a stock_minimo; estos productos se listan en GET /api/admin/inventario/bajo-stock.
- Las compras y ventas manejan un estado (PENDIENTE, COMPLETADA, ANULADA); solo el módulo de Compras puede cambiar el estado de una compra.
- Las bajas de usuarios, productos, proveedores y clientes son lógicas (campo activo), preservando el histórico de compras, ventas y movimientos asociados.
- La factura de una venta se genera bajo demanda en formato PDF a través de GET /api/admin/ventas/{id}/factura.

CONFIGURACIÓN DE ENTORNO

Perfil local con las credenciales de base de datos y la configuración del JWT:

- jwt.secret: clave secreta en Base64 usada para firmar los tokens (no debe subirse a control de versiones).
- jwt.expiration: tiempo de vida del token en milisegundos (3 600 000 ms = 1 hora).

Migraciones de base de datos

El esquema se crea y evoluciona mediante scripts versionados en src/main/resources/db/migration:

- V1__initial_schema.sql: creación de las doce tablas del modelo de datos.
- V2__load_initial_data.sql: carga de datos iniciales (áreas/roles, categorías, productos, etc.).

Flyway ejecuta estos scripts automáticamente al iniciar la aplicación, en orden de versión, y registra el historial aplicado en su tabla de control interna.

ARRANCAR EN BACKEND

- Instalar y tener disponible PostgreSQL; crear la base de datos minierp.
- Configurar las credenciales de conexión en application-local.properties (o mediante variables de entorno).
- Compilar y ejecutar el proyecto (por ejemplo, con el wrapper de Maven: ./mvnw spring-boot:run).
- Al iniciar, Flyway aplicará automáticamente las migraciones y dejará la base de datos lista con la información inicial.
- El backend quedará escuchando en http://localhost:8080.
- Configurar el frontend Angular para apuntar a esa URL base y ejecutarlo en http://localhost:4200 (origen habilitado en CORS).